Synergy

Progetto programmazione III

Eddy Pasquale Chiacchio
Pasquale Falco

Scopo

Punti Chiave

- Centralizzazione dei workflow aziendali
- Miglioramento della tracciabilità delle attività
- Facilitazione nella condivisione dei documenti

Design Pattern

Singleton

Factory Method

Strategy

Composite

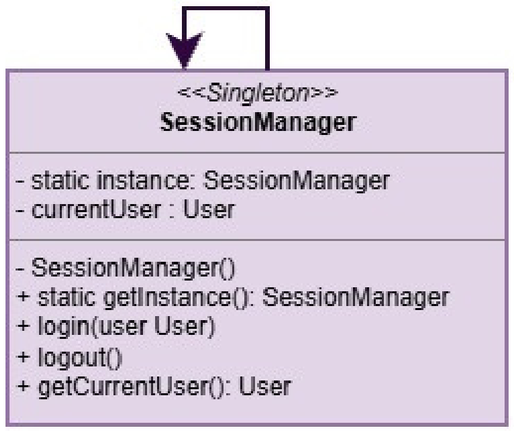
Observer

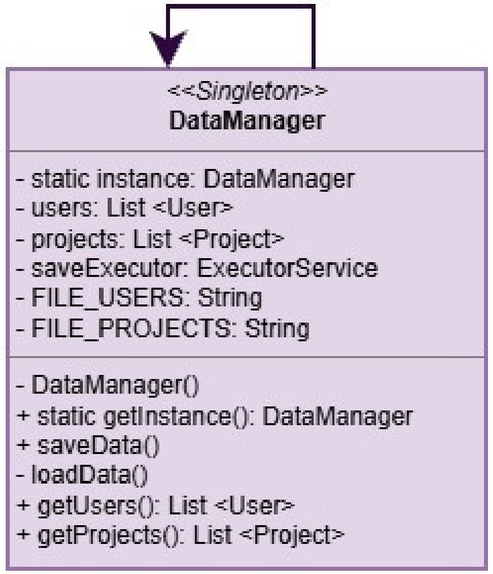
Singleton

Scopo
Garantire la gestione centralizzata della persistenza dati e il controllo univoco dello stato della sessione utente.
Componenti:
Classi DataManager e SessionManager, entrambe implementate con un'istanza statica privata e un costruttore privato per impedirne l'istanziazione esterna.
Funzionamento
Il pattern assicura che in ogni momento dell'esecuzione di Synergy esistano esclusivamente un'unica istanza del gestore dati e un'unica istanza del gestore sessione. Questo previene conflitti nell'accesso ai file di dati e garantisce che l'identità dell'utente correntemente loggato sia sincronizzata in tutti i componenti del sistema.
Dettagli Tecnici:

- Entrambe le classi espongono il metodo statico getInstance().
- DataManager: gestisce le liste globali users e projects, coordinando il caricamento e il salvataggio dei dati.
- SessionManager: mantiene il riferimento all'attributo currentUser, gestendo le operazioni di login() e logout().

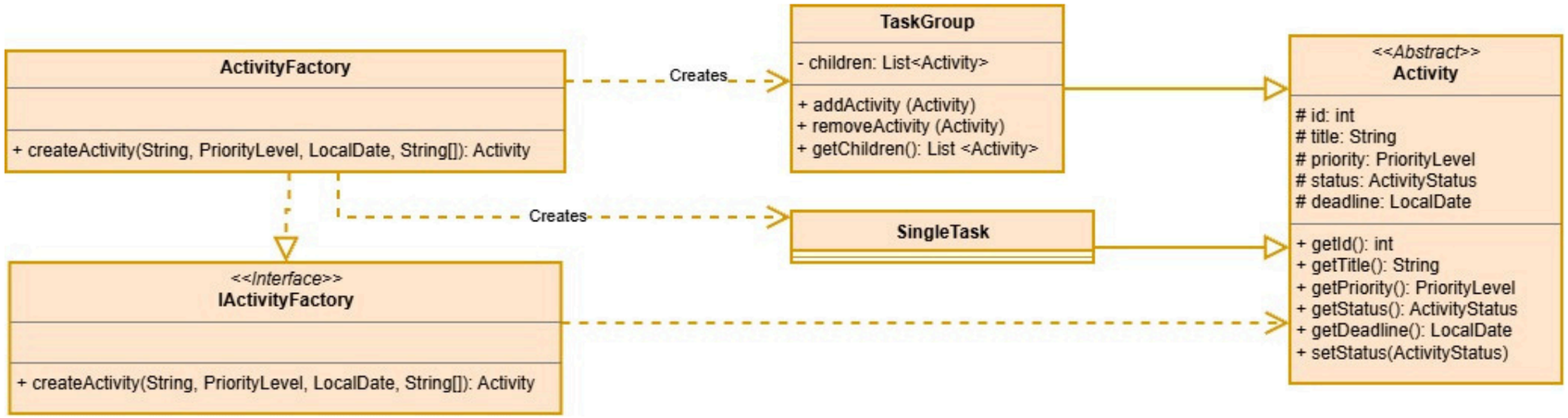
Utilità
Fornisce ai vari controller un punto di accesso globale: il DataManager permette la manipolazione dei dati di sistema, mentre il SessionManager consente di verificare i permessi dell'utente attivo in ogni operazione.





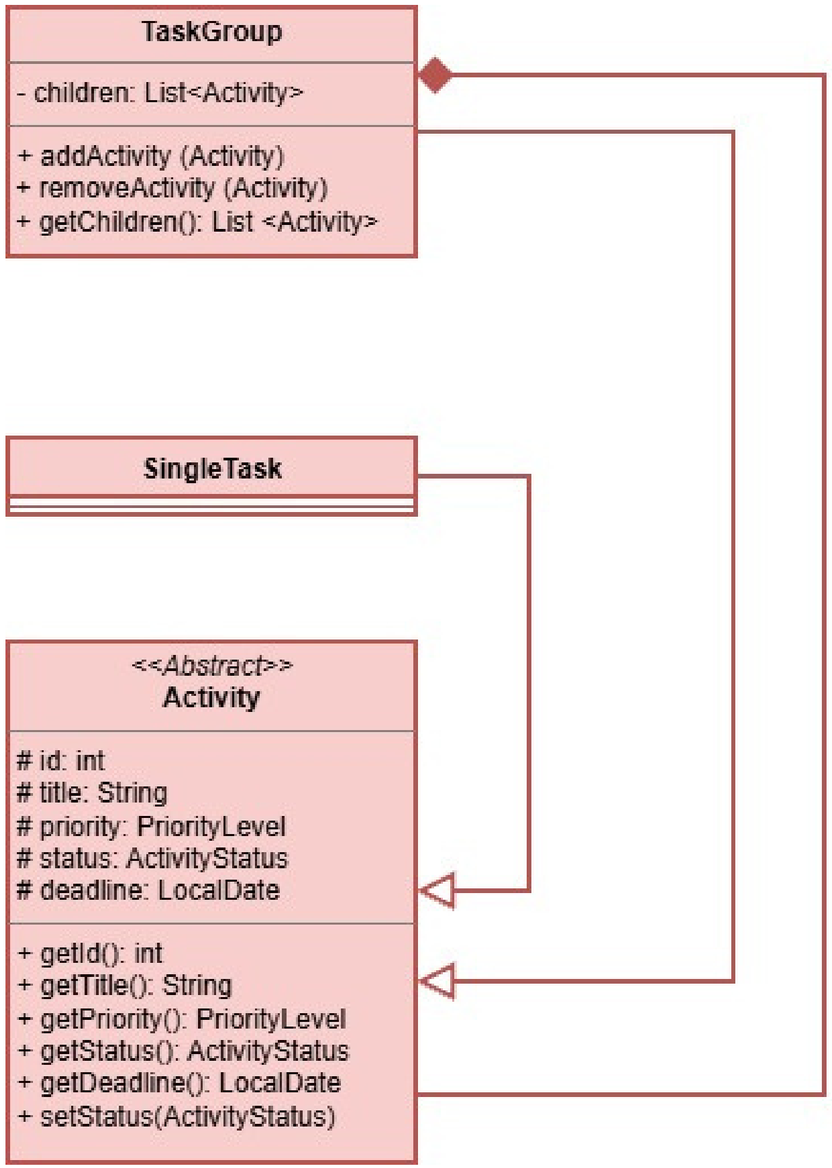
Factory Method

Scopo
Disaccoppiamento della logica di creazione delle attività.
Componenti:
IActivityFactory(Factory), ActivityFactory(Concrete Factory), le classi prodotto SingleTask e TaskGroup(ConcreteProduct) e Activity(Product).
Funzionamento:
Centralizza la creazione degli oggetti Activity. Il client richiede un'attività e la Factory decide quale classe concreta istanziare in base ai parametri passati.
Dettagli Tecnici:
Il metodo createActivity(...) incapsula la logica di istanziazione.
Utilità
Permette di aggiungere nuovi tipi di task (es. task ricorrenti) in futuro senza dover modificare il codice dei Controller che le utilizzano.



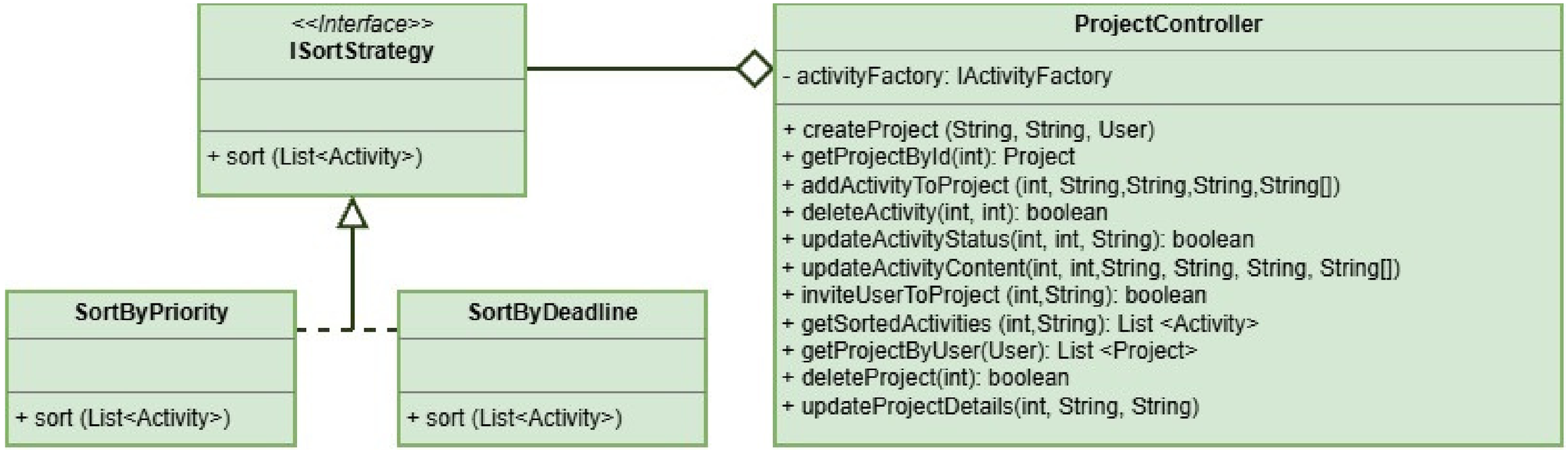
Composite Pattern

Scopo
Gestione Gerarchica tra le attività singole (Single Task) e le sotto-attività (Task-Group).
Componenti:
Activity(Component), SingleTask(Leaf) e TaskGroup (Composite).
Funzionamento:
Il TaskGroup contiene una lista di oggetti di tipo Activity. Poiché sia SingleTask che TaskGroup ereditano da Activity, possono essere trattati uniformemente dal sistema.
Dettagli Tecnici:
Relazione di aggregazione ricorsiva: la classe TaskGroup ha un attributo children: List<Activity>.
Utilità
Fondamentale in Synergy per permettere ai Project Manager di creare strutture di progetto complesse con attività annidate.



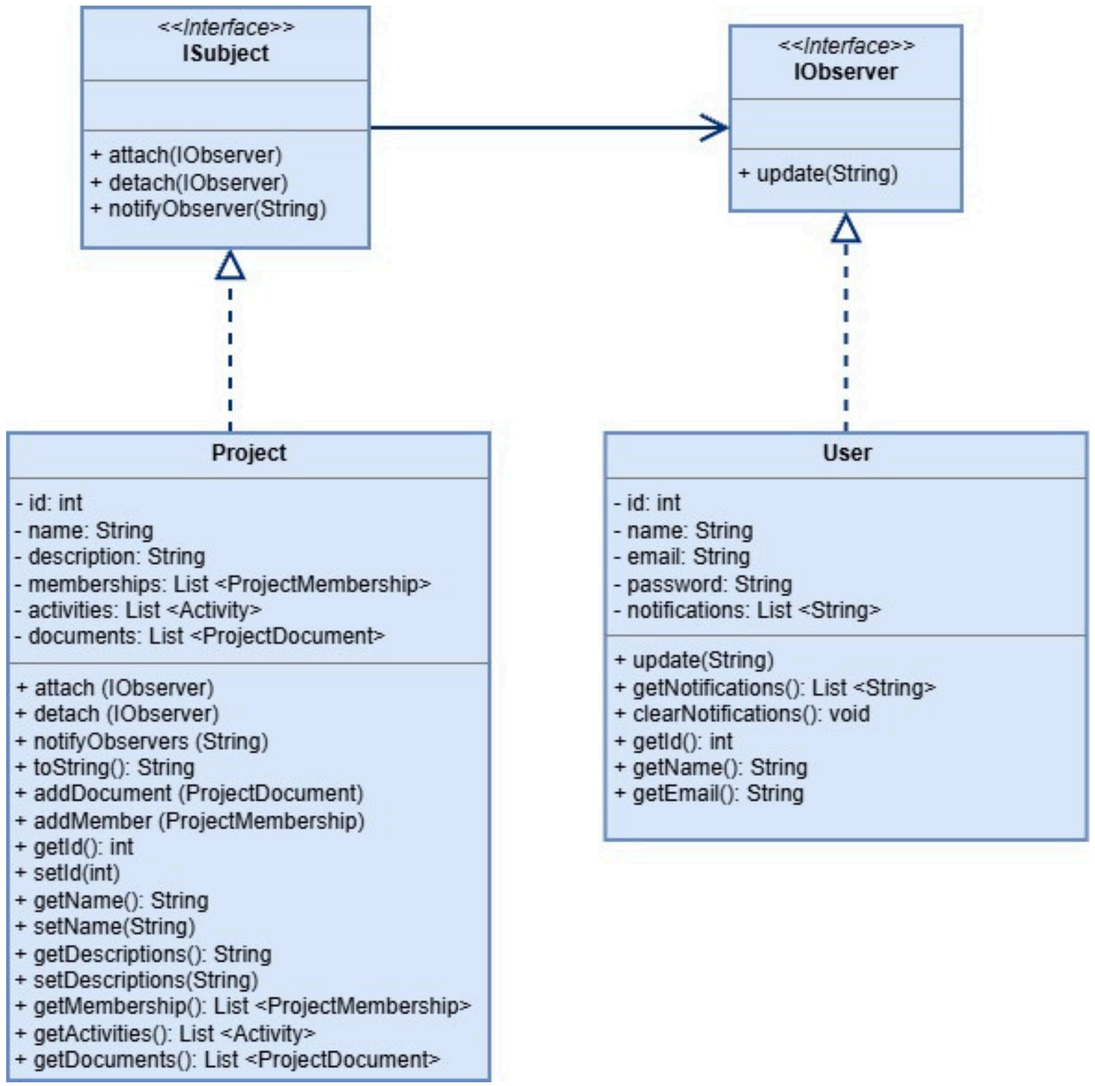
Strategy

Scopo
Definizione di algoritmi di ordinamento intercambiabili per la Dashboard.
Componenti:
ProjectController(Context), interfaccia ISortStrategy(IStrategy) e classi concrete SortByPriority e SortByDeadline(ConcreteStrategy).
Funzionamento:
Il ProjectController delega l'operazione di ordinamento a un oggetto "strategia". L'algoritmo può essere cambiato a runtime in base alla scelta dell'utente.
Dettagli Tecnici:
Il metodo sort(List<Activity>) viene implementato in modo differente nelle classi concrete per filtrare i dati.
Utilità
Permette una visualizzazione dinamica e personalizzata delle attività senza appesantire la logica del Controller principale.



Observer

Scopo
Gestione del sistema di notifiche push e aggiornamenti in tempo reale.
Componenti:
Interfaccia ISubject(Observable), Project(ConcreteObservable), interfaccia IObserver(Observer) e User(ConcreteObserver).
Funzionamento:
Gli utenti si "iscrivono" a un progetto. Quando avviene una modifica (es. nuovo documento o creazione task), il progetto notifica automaticamente tutti gli osservatori.
Dettagli Tecnici:
Metodi attach(), detach() e notifyObserver() nel Subject; metodo update(message: String) nell'Observer.
Utilità
Garantisce che i Team Member siano sempre aggiornati sull'avanzamento del lavoro senza dover ricaricare manualmente la pagina.



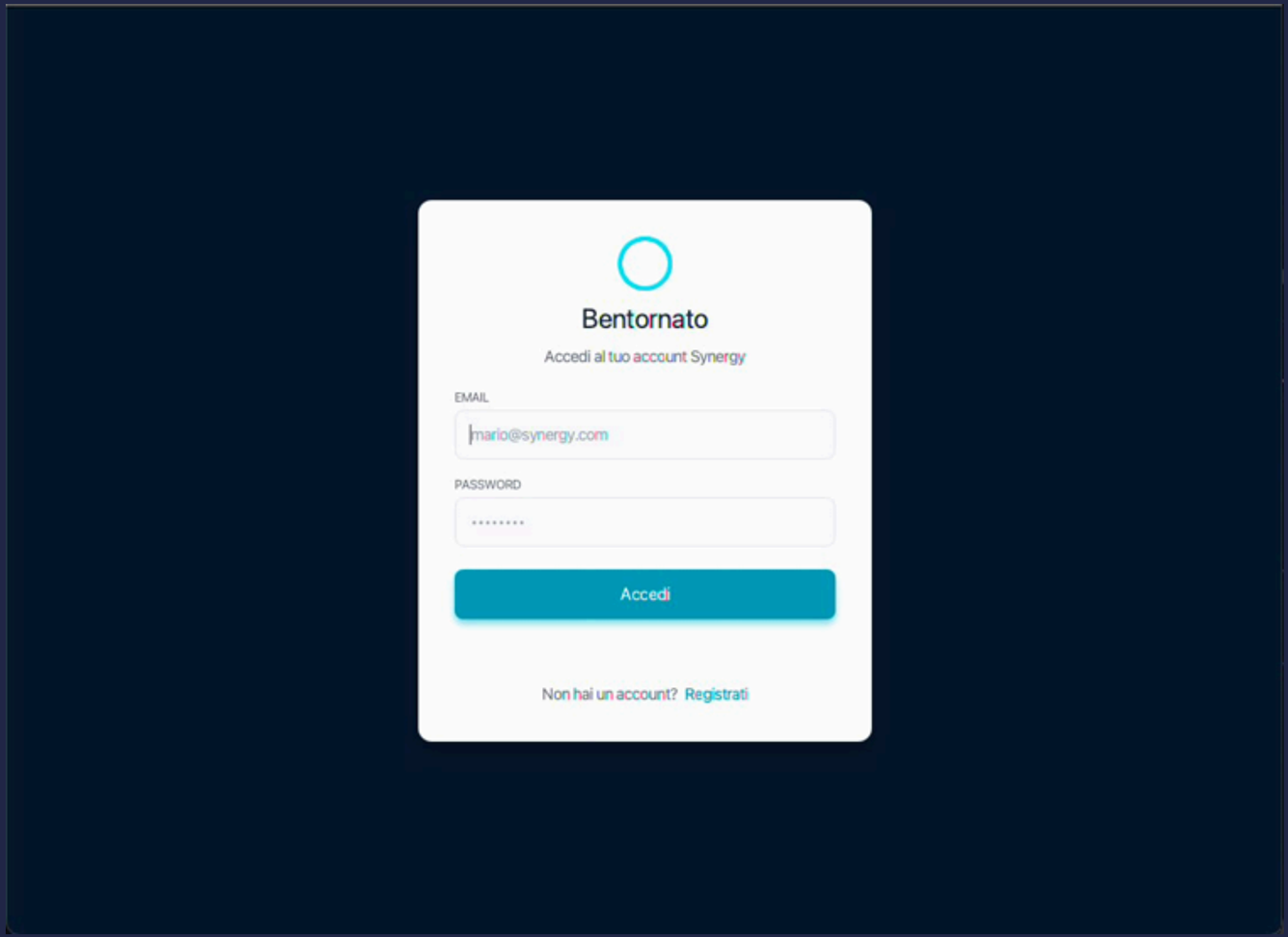
Tecnologie Utilizzate

Linguaggio
JAVA

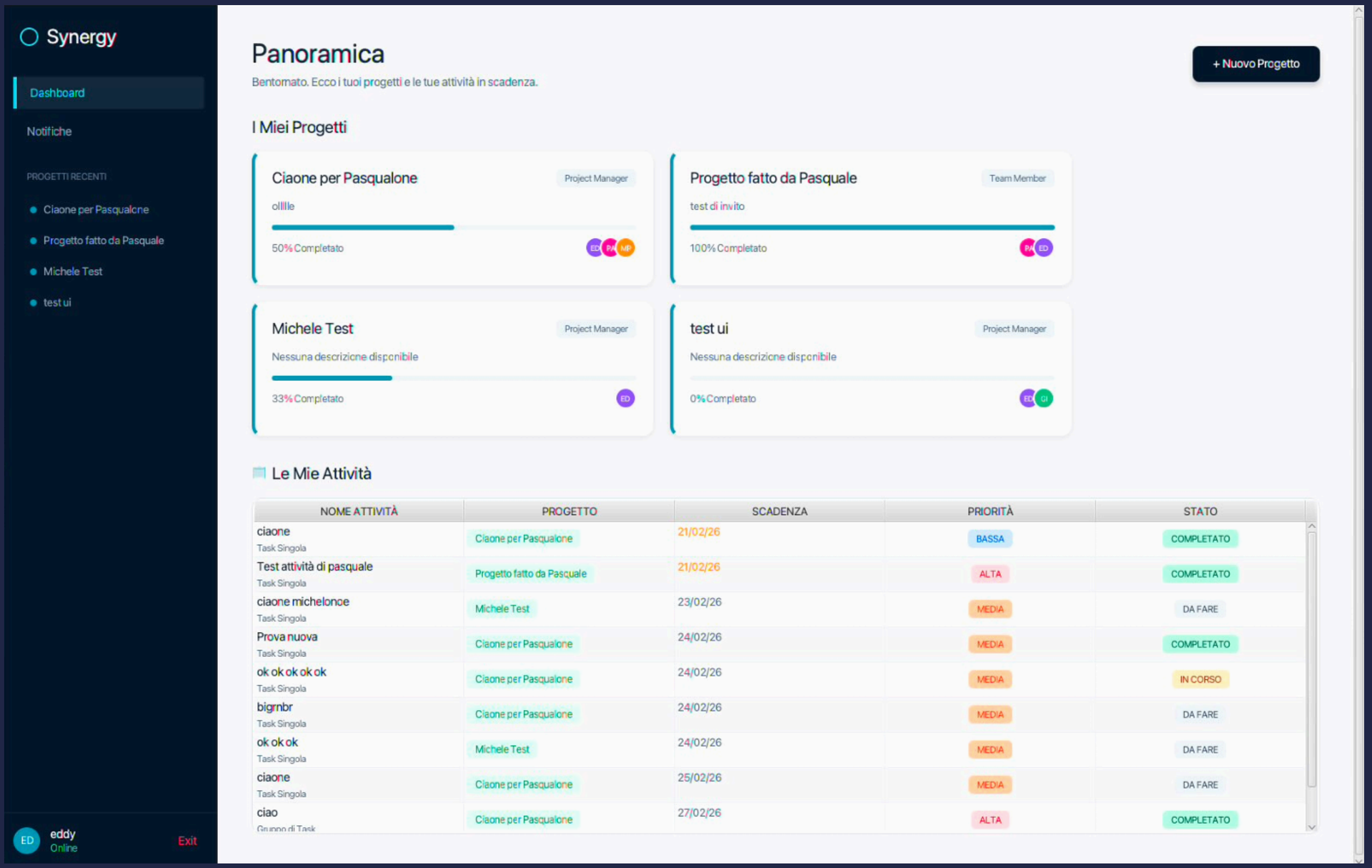
Ambiente di Sviluppo
ECLIPSE & SCENE BUILDER

Interfaccia Utente
JAVAFX & FXML

Interfaccia LOGIN



Interfaccia DASHBOARD



Interfaccia PROJECT

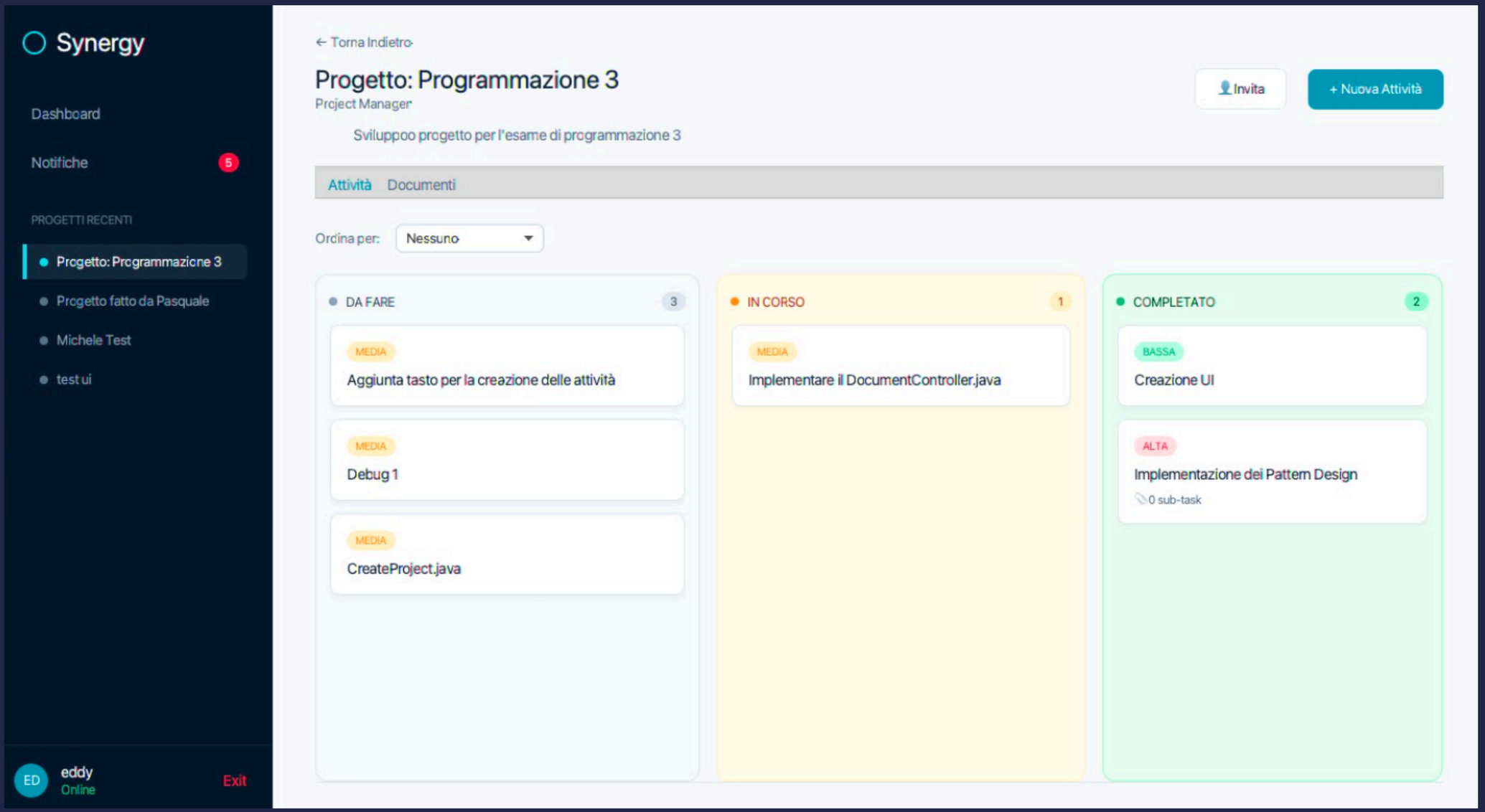
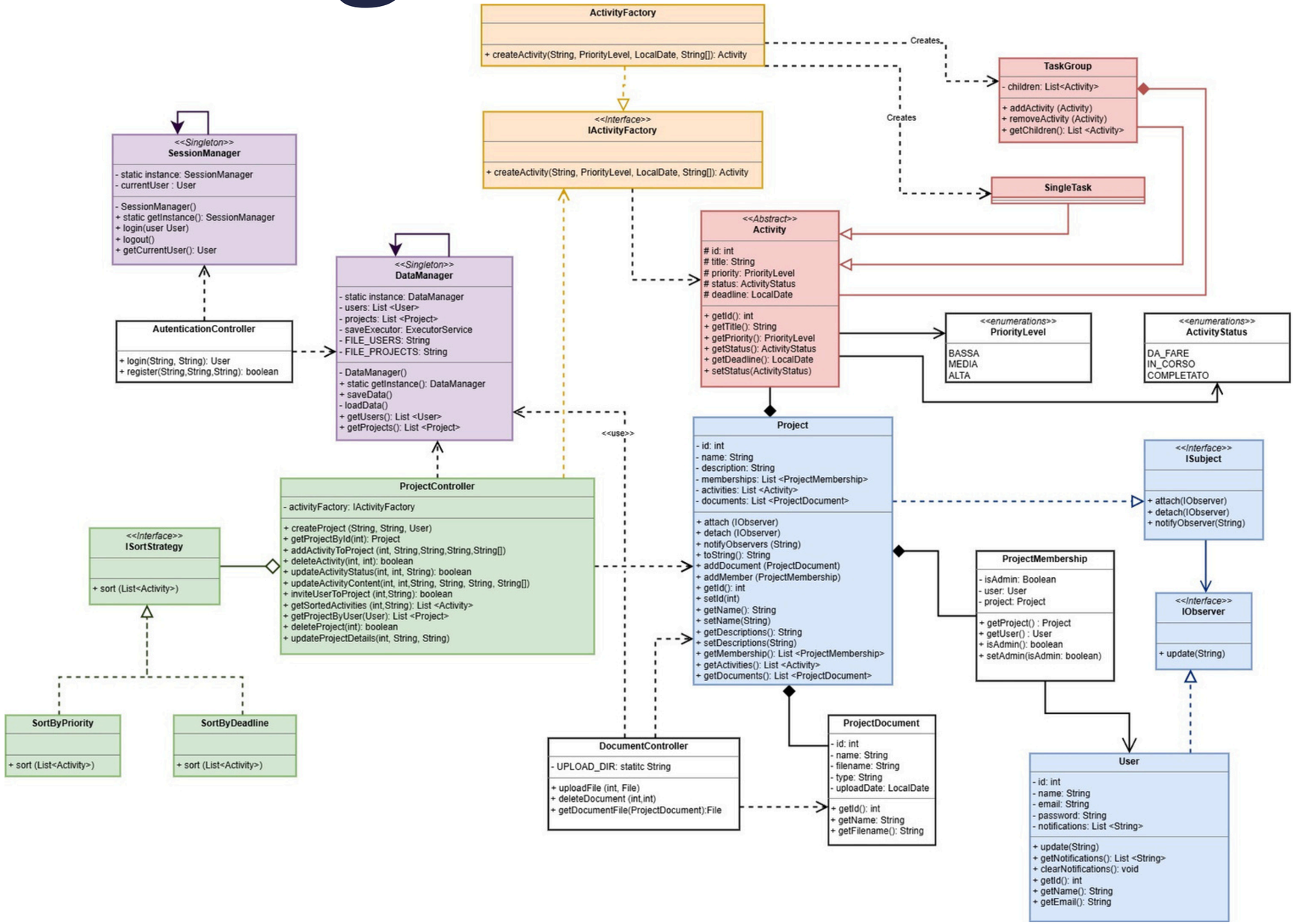


Diagramma UML



Conclusioni

La piattaforma Synergy aiuta i team e i Project Manager a coordinare attività semplici o progetti complessi, ottimizzando la gestione dei flussi di lavoro e delle scadenze. In sintesi, il sistema mira a migliorare l'efficienza collaborativa in ambienti aziendali o accademici, facilitando la tracciabilità delle informazioni, la condivisione documentale tra i membri del team.

GRAZIE
PER
L’ATTENZIONE